

RAP 後端開發練習 2：CDS Interface View 資料模型基礎 + Metadata Extension (UI Annotation) 入門

Lecture

Part A：資料模型——從 DDIC Table 到 CDS Interface View

rap01 講過 RAP 的五層架構，這一課動手蓋前兩層：**DDIC Table**（實際存資料）跟 **CDS Interface View**（資料模型，之後 rap03 的 Behavior Definition 會直接綁在這個 View 上）。

Eclipse ADT 建立 Transparent Table：Step by Step

在看 DDL 內容之前，先講怎麼從零在 Eclipse ADT 建出這個 Table 物件的空殼（☒ 已依使用者實測畫面校正：對著套件或既有物件按右鍵，選單裡直接有 **New Database Table** 這個項目，不需要繞經 **Other ABAP Repository Object** 精靈——這個系統的 ADT Plugin 把常用物件類型都內建成右鍵選單的直接捷徑，跟建 Index、rap04 建 Service Binding 是同一套模式）：

1. 在 **Project Explorer** 展開你的 ABAP Project，找到套件 **\$TMP**（本課程一律用這個套件），或直接在既有物件上按右鍵也會有同樣的捷徑。
2. 對著 **\$TMP** 按滑鼠右鍵 → 直接選 **New Database Table**。
3. 填寫：
 - **Project**：維持預設
 - **Name**：**ZRAP02_TASK**
 - **Description**：**RAP02 Task Root Table**
 - **Package**：**\$TMP**
 - 按 **Next**（**\$TMP** 套件會跳出「Select Transport Request」畫面，比照 rap04 已經記過的做法——畫面顯示「No change recording enabled for package \$TMP」時，什麼都不用選，直接按 **Finish** 即可）
4. 精靈跑完會自動開啟編輯器，系統會帶出一個最小骨架（通常只有 **mandt** 這個 Key 欄位加一個預留欄位），把它整個改寫成下方「**DDIC Table 設計要點**」列出的完整內容即可，不用手動一行一行修骨架。
5. 按 **Ctrl+S** 存檔，再按工具列的 **Activate**（或 **Ctrl+F3**）啟用。
6. 想確認表格有沒有真的建出來、資料長怎樣，可以對著這個物件按右鍵 → **Open With** → **Data Preview**（此時應該是空的，還沒有任何資料列）。

DDIC Table 設計要點（沿用 [.claude/rules/sap-adt-mcp.md](#) 第 34 / 39 節已經驗證過的 annotation 慣例）：

```
@EndUserText.label : 'RAP02 Task Root Table'
@AbapCatalog.enhancementCategory : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #LIMITED
define table zrap02_task {
    key client : mandt not null;
    key task_id : zrap02_taskid not null;
```

```

description : text100;
status      : zrap02_status;
priority    : zrap02_priority;
due_date    : zrap02_duedate;
created_at   : timestamp1;
created_by   : syuname;
}

```

兩個容易寫錯的地方（這次出題實際踩到、修正過的）：

- **timestamp1** 不能寫成 **abap.timestamp1**——CDS 的 **abap.*** 開頭是內建型別關鍵字（**abap.char** / **abap.dats** / **abap.int4** 這些），但 **TIMESTAMP1** 其實是一個標準 **Data Element**，要用 DDIC 物件引用的寫法（不加 **abap.** 前綴，直接寫 **timestamp1**，就像 **mandt** / **syuname** 一樣）。這兩種型別來源（CDS 內建 vs DDIC Data Element）語法看起來很像，容易搞混，寫錯的錯誤訊息是籠統的 **Can't save due to errors in source**，不會明確告訴你是這裡的問題。
- **created_at** / **created_by** 現在還用不到——這兩個欄位是先幫 rap05（Determination）鋪路的，讓 Managed BDEF 在建立資料時自動填入時間戳跟建立者，這一課先建好欄位，實際自動填值的邏輯留到 rap05。

DDL 語法深入：key / not null / Foreign Key / Currency-Quantity 欄位

- **key / not null**：key 標記這個欄位是主鍵的一部分，Key 欄位隱含 **not null**（不用重複寫，但寫了也不會錯，**ZRAP02_TASK** 沿用官方慣例明寫出來）；非 Key 欄位如果沒寫 **not null**，代表允許 initial 值（ABAP 沒有真正的 NULL 概念，空值就是該型別的 initial value，例如 CHAR 是空白、DEC 是 0）。
- **Foreign Key**（[.claude/rules/sap-adt-mcp.md](#) 第 10 節已經驗證過的語法，**WITH FOREIGN KEY** 子句是同一個欄位宣告的一部分，中間不能有分號）：

```abap klasse : ztr21_klasse with foreign key [0..*,1] ztr21_class where mandt = ztr21_stud.mandt and klasse = ztr21_stud.klasse; ``` [n,m] 是 cardinality：n（外鍵表這側）通常是 1 或 [0..1]；m（檢查表這側）通常是 1、[0..1]、[1..\*] 或 [0..\*]——多筆明細對應 1 筆表頭（例如多個學生屬於同一個班級）就是 [0..\*,1]。⚠️ **DDIC 外鍵只在畫面輸入（SM30 / Dynpro）層級生效，不是資料庫層的強制約束**——Open SQL 用 INSERT/MODIFY 塞一個檢查表沒有的值一樣會成功，這一點分享給大家記住，跟第 A2 節提到的 Domain 固定值清單「只在 UI 層生效」是同一個道理。

- **欄位型別：Built-in Type vs Data Element**：下一節（Part A2）會完整講規則，這裡先總結語法差異——**abap.char(10)** / **abap.dats** / **abap.int4** 這種帶 **abap.** 前綴的是 CDS 內建型別關鍵字；**mandt** / **syuname** / **timestamp1** / **zrap02\_taskid** 這種不帶前綴、直接寫名稱的，是引用一個 DDIC **Data Element**。
- **金額 / 數量欄位要指定 Currency / Unit**：CDS 的金額欄位（**DEC** / **CURR** / **DECFLOAT\*** 型別）跟數量欄位（**QUAN** / **DECFLOAT\*** 型別）都需要一個「參考欄位」告訴系統這個數字的單位是什麼，用 **@Semantics.amount.currencyCode** / **@Semantics.quantity.unitOfMeasure** annotation 指定：

```
``abap @Semantics.amount.currencyCode: 'currency' price : s_price;
```

```
@Semantics.quantity.unitOfMeasure: 'unit' qty : s_quan;
```

`currency : s_currcode; unit : s_unit; ``` 第 39 節已經踩過這個坑：如果一個 **View** 裡有兩個金額欄位卻只給一個共用的貨幣欄位、沒有明確用 **currencyCode** annotation 指向它，啟用會報（specify reference table

AND reference field)——系統沒辦法自動判斷該用哪個貨幣欄位去換算，**annotation** 就是用來消除這個模糊性的。ZRAP02\_TASK 目前沒有金額 / 數量欄位，這裡先建立觀念，用得到時（例如以後想加「預估工時」這種數量欄位）就知道該怎麼寫。

## Primary / Secondary Index：概念與 Eclipse ADT 建立步驟

- **Primary Index**：由 Table 的 Key 欄位自動組成，每張表**恰好一個**，建表當下系統自動建立，不用自己管。
- **Secondary Index**：針對「常出現在 WHERE 條件、但不是 Key 欄位」的欄位額外建的加速結構，資料庫的 Optimizer 會自己判斷要不要用（不是你 WHERE 用了這個欄位就一定會用到 Index）。經驗法則：
  - 適合建在「資料量大、選擇性高（篩選後只剩一小部分資料）」的欄位組合上
  - 不要每個查詢情境都建一個 Index——每多一個 Index，INSERT/UPDATE 就要多維護一份索引結構，是有成本的
  - 已經是 Key 欄位（已經在 Primary Index 裡）的欄位，不需要重複建 Secondary Index

⚠ ⚠ 已實測確認：Eclipse ADT 的「New Table Index」精靈在這系統上對長表名走不通，要改用 SE11——這是一路排錯才確認的結論，過程值得記錄：

1. 對著 Table 按右鍵，選單裡**直接就有 New Table Index** 這個項目（不需要繞經 Other ABAP Repository Object 精靈，這點跟 rap04 記過的 Service Binding 一樣是這個系統內建的右鍵捷徑）。
2. 但精靈裡的 **Name** 欄位長度上限只有 **10 個字元**，而且這個欄位的預設值就是你右鍵點的那張表的名稱（例如對著 ZRAP02\_TASK1 按右鍵，Name 預設就是 ZRAP02\_TASK1）——問題是 ZRAP02\_TASK1（12 碼）、ZRAP02\_TASK（11 碼）都已經超過 10 碼上限，這代表這個欄位背後其實是把「輸入值」當成「要建索引的表名」在用，但這個系統的表名慣例（ZRAPnn\_<實體>）幾乎必然超過 10 碼，導致這個欄位從一開始就填不進正確的表名。
3. 如果勉強改填一個 10 碼以內、但不是真表名的名稱（例如 ZRAP02IX1）硬過這一關，精靈後續會切到一個內嵌的傳統 Create/Change Index 畫面（標題「Dictionary: Change Index」），這個畫面的 Table Name 欄位會顯示你剛才填的那個假名稱且完全鎖住不能改，之後點 Table Fields 選欄位時會報 Table <假名稱> is not active in ABAP Dictionary——證實這條路徑走進死巷，沒有辦法回頭指定真正的表。
4. 結論：改用 SAP GUI 的 SE11 交易碼直接建，完全繞過這個 ADT 精靈：
  1. SE11 → Database table 輸入 ZRAP02\_TASK（或你練習用的 ZRAP02\_TASK1）→ Display。
  2. 選單 Goto → Indexes（或畫面上的 Indexes 按鈕），進入這張表的索引清單。
  3. Create → Index ID 填一個 3 碼英數字（例如 001；因為是自己的 Z 表不是 SAP 標準表，Index ID 不能用 Y / Z / J / H 開頭，數字最安全）。
  4. 填 Short Description，在欄位清單勾選 STATUS、DUE\_DATE。
  5. 存檔（\$TMP / 所屬套件通常不需要傳輸請求）→ Activate。
  6. 存好之後畫面會顯示 Status: Active / Saved，Index <表名>~<ID> exists in database system HDA 之類的確認訊息。

如果你的表名剛好 10 碼以內，理論上 ADT 精靈那條路徑應該走得通（畢竟 bug 的根源是「表名超長」），但這個系統的 ZRAPnn\_ 命名慣例幾乎不可能符合，所以本課程一律建議直接用 SE11，不要浪費時間在 ADT 精靈上。

用 SQL Console 的 EXPLAIN PLAN 確認 Index 有沒有被用到——⚠ 這系統做不到，已確認是限制

⚠️ ⚠️ 已實測確認：這系統的 **SQL Console** 沒有內建 **Explain Plan** 功能，這一步無法執行，記錄一下排查過程跟結論：

1. 開啟 **SQL Console** 的正確路徑（已校正）：在 Project Explorer 要對著最上層的 **Project** 節點（例如 `S4H_130_monica_en`）按右鍵才會看到 **SQL Console** 這個選項——對著 `$TMP` 套件或帳號節點按右鍵沒有這個選項。
2. 在 SQL Console 輸入查詢（例如 `SELECT * FROM zrap02_task1 WHERE status = 'O' AND due_date < '20261231'`）。
3. 依序找過三個地方都沒有 **Explain Plan** 選項：工具列 **Run** 按鈕旁邊的下拉箭頭、最上方選單列的 **Run**、在查詢文字上按右鍵——都只有 **Check / Run**（F8，真的會執行查詢）這兩個選項；**Run As** 子選單展開後顯示（`none applicable`）。
4. 結論：這個系統的 ADT SQL Console 是比較輕量的版本，沒有內建 **Explain Plan** / 執行計畫分析功能——這種功能通常要靠獨立的 **SAP HANA Database Explorer**（另一個工具 / Perspective，不是 ADT 內建的 SQL Console），這系統目前沒有配置這個工具，這一步只能跳過。
5. 這一步跳過不影響練習 1 的驗收——Secondary Index 建立成功與否，用 SE11 索引維護畫面顯示的 **Status: Active / Saved** 加上欄位清單正確（**STATUS / DUE\_DATE**）就已經是足夠的證據，**Explain Plan** 只是「錦上添花」的延伸驗證，不是必要條件。

概念補充（沒辦法在這系統實際驗證，但觀念要知道）：如果你的系統有 **Explain Plan** 功能，看執行計畫時要注意——就算索引建對了，Optimizer 也不一定會用到它，常見原因：

- 表格資料量太小，Optimizer 判斷全表掃描比用 Index 更快（`$TMP` 訓練用的表通常資料筆數很少，很可能就是這個原因）
- **WHERE** 條件跟 Index 定義的欄位順序/組合對不上
- 資料庫統計資訊還沒更新

## 練習 1：自己建一個 **Secondary Index**

輪到你了：對著 `ZRAP02_TASK` 用 SE11 依照上面的步驟自己建一次 **Secondary Index**（可以用一樣的 `status+due_date` 組合，也可以自己想一個查詢情境）。這系統的 SQL Console 沒有 **Explain Plan** 功能（上面已經確認），驗收依據是 SE11 索引畫面顯示 **Status: Active / Saved**。完成後跟我說一下建立過程跟最終狀態，我會幫你核對。

Part A2：欄位型別一律要引用 **Data Element**，不要留 `abap.char()` 這種內建型別（硬性規則）

`.claude/rules/abap-style.md` 已經記錄了一條硬性規則：新建的 **DDIC** 物件，任何欄位如果在語意上對應到標準表的既有欄位（例如物料號碼對應 `MATNR`），一律要引用那個標準 **Data Element**，不可以自己用 `abap.char(...)` 重新定義一個長度剛好一樣的版本；如果真的是專案/課程獨有的業務概念、找不到對應的標準 **Data Element**，也不能偷懶留著內建型別，要自己建一組 **Domain + Data Element**。

這一課的五個非 Key 欄位剛好示範了兩種情況：

① 有現成標準 **Data Element** 可以直接重用——`description` 改用 `TEXT100`（`CHAR100`，套件 `BK`，語意中立、沒有綁定特定模組的通用文字欄位，用 ADT quickSearch 查證過，不是憑記憶猜的）。曾經考慮過 `priority / due_date / status` 有沒有現成標準 DE，quickSearch 查到的候選（如 `/SAPAPO/PRIORITY`、`GM_TASK_STATUS`）全部是特定模組專用（`APO`、`Grants Management`），硬套會誤導欄位語意，所以判定「沒有真正合適的標準 DE」，改走第二種情況。

② 沒有合適標準 DE，自己建 Domain + Data Element——task\_id / status / priority / due\_date 各自建了一組：

Domain	固定值	Data Element	標籤
ZRAP02_TASKID	無 ( 自由輸入 CHAR10 )	ZRAP02_TASKID	Task ID
ZRAP02_STATUS	O=Open / D=Done	ZRAP02_STATUS	Status
ZRAP02_PRIORITY	H=High / M=Medium / L=Low	ZRAP02_PRIORITY	Priority
ZRAP02_DUEDATE	無 ( DATS 日期 )	ZRAP02_DUEDATE	Due Date

status / priority 這兩個特別示範了 Domain 固定值清單的建立語法 ( .claude/rules/sap-adt-mcp.md 第 8 / 25 節記載的模式：doma:fixValue 只給 doma:low + 遞增 doma:position，不帶 doma:high，這種離散單值型的 doma:text 說明文字會正確存下來 )。

⚠ 一個重要的觀念澄清：Domain 固定值清單只在畫面/UI 層級生效 ( Fiori Elements 會自動把它變成 F4 下拉選單 )，不是資料庫層的強制約束——程式用 INSERT/MODIFY 硬塞一個不在清單裡的值一樣會成功。也就是說，就算 status 已經有 Domain 固定值限制成只能是 O/D，rap06 的 Validation 邏輯還是必要的，Domain 給的是「使用者操作畫面的友善提示」，Validation 才是「真正擋掉非法資料」的防線。

⚠ Data Element 建立也踩到一個熟悉的坑：POST 建立 Data Element 時，即使 XML 裡帶了完整的 shortFieldLabel / mediumFieldLabel 等標籤，第一次 POST 回應雖然 201 成功，但標籤實際上沒有真的存進去 ( 跟 .claude/rules/sap-adt-mcp.md 第 8 節記載的「DE 標籤不會在 POST 當下落地」完全一致 ) ——一定要再走一次 LOCK→PUT ( 同一份 XML ) →UNLOCK，PUT 這一步才會真的把標籤寫進去，光靠 POST 不夠。

CDS Interface View：

```
@AbapCatalog.sqlViewName: 'ZIRAP02TASK'
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'RAP02 Task Interface View'
@ObjectModel.compositionRoot: true
@Metadata.allowExtensions: true
define root view ZI_RAP02_TASK
 as select from zrap02_task
{
 key task_id,
 description,
 status,
 priority,
 due_date,
 created_at,
 created_by
}
```

四個必要 annotation，各自的作用：



Annotation	作用
<code>@AbapCatalog.preserveKey: true</code>	rap01 已經踩過的坑——這個系統的 RAP 根 View 缺了這行會直接啟用失敗 ( " <code>@AbapCatalog.preserveKey: true</code> " missing for entity in BO structure )
<code>@ObjectModel.compositionRoot: true</code>	告訴 RAP 框架「這是一個獨立的 Business Object 根節點」，不是被其他 View 組合進去的子節點
<code>@AccessControl.authorizationCheck: #NOT_REQUIRED</code>	先關掉權限檢查，讓這一課專心在資料模型上；正式的 RAP 授權機制是進階主題，不在本課程範圍
<code>@Metadata.allowExtensions: true</code>	這一課新增的一行——沒開這個，Part B 的 Metadata Extension 會建不起來 ( 系統會說找不到可擴充的 View )

⚠ `@AbapCatalog.sqlViewName` ( 程式碼區塊第一行 ) ——DDL View Name 跟 SQL View Name 是兩個不同名字，用途也不一樣：

- **DDL View Name** ( `define view ZI_RAP02_TASK` 這裡的名稱 )：這是在 Eclipse ADT / ABAP 原始碼 / CDS Association / BDEF 裡實際引用這個 View 用的「邏輯名稱」，也是 Repository 物件的正式名稱 ( Transport、Where-Used、quickSearch 查的都是這個 )，長度上限是一般 Z 物件的 **30 碼**。
- **SQL View Name** ( `@AbapCatalog.sqlViewName` annotation 指定的值，例如 '`ZIRAP02TASK`' )：這是這個 View 在資料庫底層真正建出來的**實體 SQL VIEW 物件名稱**，官方文件明講這是「純技術用途的輔助物件」 ( "purely technical helper construct" )，不應該被直接引用。長度上限只有 **16 碼**——這是沿襲自 ABAP Dictionary 資料庫物件命名規則 ( Table / View 的實體名稱都是這個上限，官方文件原文：「maximum of 16 characters」 )。
- 為什麼要分兩個名字：DDL View Name 為了可讀性可以取到 30 碼 ( 例如練習 2 的 `ZI_RAP02_FLIGHT_PRACTICE` 就有 24 碼 )，但資料庫底層物件名稱的上限比較短，所以這個系統用的舊式 `define view` ( V1 ) 語法**強制要求**你額外指定一個 ≤16 碼的 SQL View Name 給底層資料庫用——這也是為什麼練習 2 的範例程式碼裡，`@AbapCatalog.sqlViewName` 給的是縮寫過的 `ZIRAP02FLPRC` ( 12 碼 )，不是跟 DDL View Name 一樣的完整名稱。
- ( 新式 `define view entity` 語法因為底層機制不同，不需要這個 annotation——這系統用不到新式語法，這裡只是補充對照，不用深究。 )

⚠ ⚠ 已依使用者實測更正：不同工具查詢 **CDS View** 用哪個名字，規則是「這個工具是不是比 **CDS** 更早就存在」，不是全部都用 **DDL Name**：

- **ABAP Open SQL**：`SELECT * FROM ZI_RAP02_TASK ...` 這樣寫，用的一律是 **DDL View Name** ( 邏輯名稱 )。官方文件明確說**直接用 SQL View Name 在 Open SQL 裡查詢是 Obsolete ( 過時 ) 用法**，從 ABAP 7.62 起在 Strict Mode 甚至**直接被禁止** ( `Direct access to a CDS-managed DDIC view (obsolete) is forbidden in strict mode` )。
- ⚠ **SE11 ( View 顯示畫面 )**：反而是用 **SQL View Name**，不是 **DDL Name**——實測畫面上查詢欄位本身就明確標示 **DDL SQL View**，要輸入 SQL View Name ( 例如 `ZIRAP02TASK` ) 才查得到；查到之後畫面上會另外顯示一個 **DDL Source** 欄位，回頭告訴你這個底層 SQL View 是哪個 CDS DDL 定義產生的 ( 顯示 `ZI_RAP02_TASK` )。原因：SE11 的 View 瀏覽功能比 CDS 更早就存在的傳統 ABAP Dictionary 機制 ( 早在 CDS 出現前，SE11 的 View 型別就是給傳統 Dictionary View / Join View 用的 )，本質上是直接對應資料庫底層那個物理物件在查——CDS View 只是後來「掛」進這套舊機制，查詢入口天生認的還是實體名稱。

- **⚠ SQ02 ( InfoSet )** 也是同樣原因，只認 **SQL View Name**——這不是 **ZI\_RAP02\_TASK** 這個 **View** 特有的狀況，任何 **CDS View** 都適用這條規則：實測輸入 DDL Name ( **ZI\_RAP02\_TASK** ) 到「Table join using basis table」欄位，直接報 **Table ZI\_RAP02\_TASK is not in ABAP Dictionary**，因為 InfoSet / SAP Query 這套工具同樣是比 CDS 更早存在的傳統機制，不認得 DDL 邏輯名稱。這一點對 **Query ( InfoSet )** 想要拿「Code to Data」的 **CDS View** 當資料來源特別重要——AMDP 課程 ( [.claude/rules/sap-adt-mcp.md](#) 第 16 節 ) 教過的「邏輯下推到資料庫執行」( Code to Data ) 這種設計，不管底層是純 CDS View 還是 CDS Table Function / AMDP，只要最終還是掛在一個 **define view ( V1 )** 物件上，在 **SQ02 / SQ01** 裡要串接時一律要用 **@AbapCatalog.sqlViewName** 那個 **SQL View Name**，不能填 **CDS DDL** 的邏輯名稱——這個限制不是這一課的特例，是 InfoSet 這套工具本身跟所有 CDS-based View 互動時的通用規則。
- **Eclipse ADT / quickSearch / Where-Used** ( 跟 Open SQL 一樣是 CDS 之後才有的現代化工具 )：用 **DDL View Name**。
- 判斷原則：這個工具是「**CDS 出現之前就存在的傳統交易碼**」( **SE11 View** 畫面、**SQ02 InfoSet.....** ) 還是「**CDS 之後才有的現代工具**」( **Eclipse ADT**、**Open SQL** )——前者認 **SQL View Name**，後者認 **DDL View Name**，沒有「全部都用同一個名字」這種簡單規則，遇到新的工具要先實測確認，不要照搬其中一種假設。

**⚠ 為什麼 View 的欄位清單裡沒有 client ( 表格裡卻有 key client : mandt not null )**：這不是漏寫，是 CDS View 的「Client Handling」自動機制——只要底層表的 Key 欄位型別是 **mandt** ( CLNT 型別的 Domain )，Open SQL / CDS 就會自動幫忙做 Client 過濾，即使 **View** 完全沒有列出這個欄位，實際查詢時系統背後還是會自動加上 **WHERE client = <目前 Session 的 Client>**，不需要自己寫、也不需要曝露在輸出欄位裡。這個行為由 **@ClientHandling.algorithm** annotation 控制，預設值就是 **#SESSION\_VARIABLE** ( 沿用底層表的 Client 欄位自動過濾 )——**ZI\_RAP02\_TASK** 沒有明寫這個 annotation，代表用的正是這個預設行為。只有故意想做「跨 Client 查詢」時，才需要明確設成 **@ClientHandling.algorithm: #NONE** 並自己把 Client 欄位加進 View 的欄位清單。

這跟 **AMDP / SQLScript** 的行為正好相反，適合對照著記：AMDP 課程 ( [.claude/rules/sap-adt-mcp.md](#) 第 16 節 ) 記過，SQLScript 直接操作 HANA 底層實體表，完全不會自動處理 Client，**SELECT \* FROM <client-dependent 表>** 撈到的是「所有 Client」的資料，必須自己手動加 **WHERE mandt = :iv\_mandt**。Open SQL / CDS View ( 自動處理 ) 跟 AMDP ( 完全不處理 ) 是這兩種資料存取技術在 Client 這件事上最大的差異，也是很多人容易搞混、誤判「怎麼 AMDP 查出來資料變多了」的根本原因。

**欄位別名與 mapping for 的取舍**：注意 View 裡的欄位 ( **task\_id**、**description.....** ) 沒有加 **as** 別名，直接沿用跟表格一樣的名稱。這是刻意的選擇——如果欄位改名 ( 例如 **task\_id as TaskID** )，BDEF 那邊建立 Managed 行為時就需要另外寫一段 **mapping for zrap02\_task { ... }** 明確對應每個欄位；欄位名稱維持一致，系統會自動對應，少一段要維護的程式碼。等以後真的有「View 欄位名稱要跟 OData 曝光的名稱不同」的需求時，才值得用別名 + mapping。

## Eclipse ADT 建立 CDS View：Step by Step

跟建 Table 一樣，先講怎麼從零建出 **ZI\_RAP02\_TASK** 這個 CDS View 的空殼：

1. 對著 **ZRAP02\_TASK** 這張 Table ( 或直接對 **\$TMP** 套件 ) 按右鍵 → **New** → **Other ABAP Repository Object...**，篩選 **Data Definition**，選取後 **Next**。
2. 填 **Name** ( **ZI\_RAP02\_TASK** )、**Description**、**Package** ( **\$TMP** )，按 **Next** → **\$TMP** 的 Transport 畫面一樣直接 **Finish**。

3. ⚠️⚠️ 接著會跳出「**Templates**」畫面，要你選一個範本——這一步選錯會直接撞上這系統的已知語法限制：

| 模板 | 語法關鍵字 | 這系統能不能用 | |---|---|---| | Define View Entity | **define view entity X as select from Y** (新式，帶 **entity**) | ❌ 不能——這系統的 ABAP 編譯器不認得 **entity** 關鍵字，啟用會直接失敗 | | Define Root View Entity | 同上，多一個 **root** 關鍵字 (給 RAP BO 根節點用) | ❌ 不能，同樣原因 | | Define View Entity with To-Parent Association | 同上，多預帶一段 **association to parent** 骨架 (給 RAP Composition 的子節點用) | ❌ 不能，同樣原因 | | **Define View (obsolete as of AS ABAP 7.57)** | **define view X as select from Y** (舊式，沒有 **entity**) | ✅ 選這個——雖然畫面標示「obsolete」，但這系統的 ABAP 版本只吃這種舊式語法，本課程從 rap01 開始寫的所有 CDS View 都是這個語法 |

選好「Define View (obsolete as of AS ABAP 7.57)」後，畫面下方會預覽這個模板的骨架內容

(`@AbapCatalog.sqlViewName / @AbapCatalog.preserveKey / define view ... as select from ...`)，確認長得跟本課程教的語法一致再按 **Next**。

(其他兩個收合的分類：**Projection View** 是給 Projection View 用的，一樣是新式 **entity** 語法，這系統也不能用；**Table Function** 是 AMDP 課程教過的 CDS Table Function，跟這裡無關，不用管。)

- 精靈通常會再問你要不要以某個既有 Table / CDS View 當範本 (Reference Object)，選 **ZRAP02\_TASK** 可以讓系統自動帶出一個包含全部欄位的初始 **SELECT** 清單，省去手動打欄位名稱。
- 編輯器開啟後，把系統產生的骨架改成上面「CDS Interface View」程式碼區塊列出的完整內容 (四個必要 annotation + 欄位清單)。
- Ctrl+S** 存檔 → **Activate**。
- 對著這個 View 按右鍵 → **Open With** → **Data Preview**，可以直接看到查詢結果 (此時應該還是空的，因為 Table 裡沒資料)。

### CDS View 分類：這個系統的命名慣例，RAP 用哪一種

實務上 (尤其是 SAP 標準物件跟坊間慣例) CDS View 依照「消費層級」分成幾種，用前綴區分，設計時通常照這個分層堆疊：

前綴	分類	用途
I_*	<b>Interface View</b> ( Basic / Composite )	純資料語意層，給其他 CDS View 或應用程式消費， <b>不帶 UI Annotation</b> ；Basic 直接對應一張表，Composite 是多個 Interface View 組合 / Join 出來的
C_* 或 P_*	<b>Consumption / Projection View</b>	疊在 Interface View 之上，加 <b>@UI.*</b> 這類畫面客製化標記，給前端 ( Fiori Elements / OData ) 消費；RAP 世代習慣用 <b>P_*</b> ( Projection )，對應 BDEF 的 <b>projection</b> 語法
R_*	Custom-Specific Reporting View	延伸層，通常給客戶自訂報表用，不在本課程範圍

**RAP 用哪一種**：這一課建的 **ZI\_RAP02\_TASK** ( **I\_\*** 前綴 ) 屬於 **Interface View**——這正是 RAP Behavior Definition 直接綁定的那一層。本課程走的是**單層模式** ( Interface View 直接掛 BDEF、也直接被 Service Definition **expose**，沒有另外疊一層 Projection View )，這是這個系統 / 這門課簡化過的做法；真實專案如果要讓同一個資料模型服務多種不同消費情境 (例如一個給後台管理用、一個給行動裝置精簡版)，才會在



Interface View 之上再疊 Projection View，各自搭配獨立的 Metadata Extension——思考題 3 已經帶過這個延伸情境。

Association vs. Join

兩者都是「把多個資料來源接起來」的手段，但語法跟執行時機不一樣：

```
-- Join：立刻把兩張表接起來，結果集直接包含兩邊欄位
define view ZI_XXX_JOIN
 as select from a
 inner join b on a.key = b.key
{
 a.field1,
 b.field2
}

-- Association：宣告一個「可能的」關聯路徑，取一個名字（慣例底線開頭）
define view ZI_XXX_ASSOC
 as select from a
 association [0..1] to b as _b on a.key = _b.key
{
 a.field1,
 _b.field2 -- 透過 path expression 「_b.欄位」使用，這裡才會真的觸發 join
}
```

Cardinality ( [min..max] ) 的可能值與個別意義 ( 查證官方文件確認，不是猜的 )：

完整語法是 `association [ [min..max] ] to target as _assoc on ...`，中括號內的 min / max 各自的規則：

寫法	意義	說明
[0..1]	零或一筆	最常見的「to-one」關聯——關聯到的資料 <b>可能不存在</b> （例如：任務可能還沒指派部門），這一課練習 2 的 <code>association [0..1] to scarr as _carrier</code> 就是這種：每筆 SPFLI 航班資料，最多只會對到一筆 SCARR 航空公司
[1..1]	剛好一筆	「to-one」但關聯對象 <b>保證一定存在</b> （例如：Foreign Key 本身不可為空、且對方資料保證存在）
[0..*]	零到多筆	「to-many」——關聯到的資料可能一筆都沒有，也可能有很多筆（例如：一個部門底下有 0~多個任務）
[1..*]	一到多筆	「to-many」但保證 <b>至少一筆</b> （例如：一張訂單至少要有一個明細項目）

規則細節 ( 官方文件明確列出 )：

- min / max 可以是 0 或正整數，max 也可以用 \* 代表「不限筆數」
- max 不能是 0 ( [0..0] 沒有意義——「保證一筆都沒有」的關聯沒有存在必要 )

- `min` 不能是 `***` ( 下限不能設成「無限多筆」 )
- `min` 可以省略，省略時預設是 0
- 如果整個 `[min..max]` 都不寫 ( 例如上面 Association 範例如果寫成 `association to b as _b on ...`，完全沒有中括號 )，系統會隱含當作 `[0..1]` ( `to-one` ) ——這是容易忽略的細節，沒寫 Cardinality 不代表「沒有限制」，而是被當成「零或一筆」
- 如果要在 `WHERE` / `HAVING` 子句裡使用這個 Association ( 第 267 行表格提過的用法 )，`max` 必須是 1 ( 也就是必須是 `to-one` 關聯，或用 Filter Condition 的 `[1: ...]` 語法強制覆寫成 `to-one` )

⚠️ ⚠️ 官方文件給的真實踩坑案例，值得完整看一次——Cardinality 只是文件性質的宣告，資料庫不會強制檢查你宣告的跟實際資料相符 ( 官方原文：「The cardinality is mainly descriptive, not prescriptive. It does not force a matching result set.」 )。如果宣告錯了，某些資料庫 ( 例如 HANA ) 會依照你宣告的 Cardinality 對產生的 `LEFT OUTER JOIN` 做效能優化 ( 幫 `to-one` 關聯加上 `TO ONE` 提示 )，宣告錯誤會導致查詢結果不完整、而且不同資料庫系統的表現還可能不一致：

" ❌ 錯誤示範 ( 官方文件原文案例 )：SCARR ( 航空公司，一筆 ) 對 SPFLI ( 航班，很多筆 )

" 是明顯的 `to-many` 關係，但完全沒寫 Cardinality，被隱含當成 `to-one` `[0..1]`

```
define view demo_cds_wrong_cardinality
as select from scarr
association to spfli as _spfli on _spfli.carrid = scarr.carrid
{
key scarr.carrid as carrid,
scarr.carrname as carrname,
_spfli.connid as connid
}
```

" ✅ 正確示範：明確宣告 `[1..*]`，符合「一家航空公司對多筆航班」的真實資料關係

```
define view demo_cds_explicit_cardinality
as select from scarr
association [1..*] to spfli as _spfli on _spfli.carrid = scarr.carrid
{
scarr.carrid as carrid,
scarr.carrname as carrname,
_spfli.connid as connid
}
```

官方文件的說明：錯誤版本在 HANA 這類會做優化的資料庫上，因為 Cardinality 被誤判成 `to-one`，產生的 Join 會被加上 `TO ONE` 提示，導致實際查詢結果比真正應該撈到的資料筆數少 ( 悄悄漏資料，不會報錯 )；正確版本明確宣告 `[1..*]`，Join 才會正確帶上 `TO MANY`，撈出真正完整的資料。這也是為什麼練習 2 一定要留意 Association 的方向：SPFLI → SCARR ( 多筆航班對一筆航空公司 ) 是 `[0..1]` / `[1..1]`；如果反過來寫 SCARR → SPFLI ( 一筆航空公司對多筆航班 )，就要用 `[0..*]` / `[1..*]`，寫反了不會啟用失敗，但查詢結果可能悄悄少掉資料，非常隱蔽的一種錯誤。

**Association vs. Join 差異總表：**

	Join	Association
何時真正執行	宣告當下 <b>立刻</b> 執行，結果集固定包含兩邊欄位	只有 <b>真的被 path expression</b> 引用才會在背後轉成 Join；沒用到完全不影響效能（不會下推到 SQL）
語法	<code>inner/left outer join b on ...</code>	<code>association [cardinality] to b as _b on ...</code>
使用方式	接完就是接完，欄位清單直接混用兩邊欄位	先「宣告一個可能的關聯」，欄位清單 / WHERE子句裡用 <b>_b.欄位</b> 才會觸發
彈性	寫死，每個消費者都被迫承擔 Join 成本	消費端自己決定要不要用，同一個宣告可以被不同查詢情境重複利用

官方文件原文：「as soon as an association is used in a path expression... it is internally transformed into a join」——這句話點出了 Association 的核心價值：先宣告「這裡有一條關聯路徑」，但把「要不要真的走這條路」的決定權留給消費端，不強迫每次查詢都付出 Join 的成本。\*RAP 的 Composition 語法（`composition [0..*] of`，rap08 會用到）本質上就是一種特殊化的 Association\*\*，用來表達 Header-Item 這種父子結構。

### Path Expression 的意義及用途

定義（官方文件）：Path Expression 是「一串用句點（.）連接起來的 CDS Association」，慣例上每一段都用底線開頭命名（`_assoc1.field`、甚至可以串更多層 `_assoc1._assoc2.field`）。上面範例的 `_b.field2` 就是最簡單的一層 Path Expression。

可以用在哪些地方（不是只有欄位清單）：

用途	範例	說明
欄位清單（曝光關聯表的欄位）	<code>_carrier.carrname</code>	最常見用法，把關聯出去的表的某個欄位直接當作自己的欄位輸出
欄位清單（曝光整個關聯，給 OData \$expand 用）	<code>_carrier</code>	不指定欄位、直接曝光整個 Association，前端可以用 OData 的 <code>\$expand</code> 語法一次把主資料跟關聯資料都撈出來——這是 RAP / Fiori Elements 常見的巢狀結構就是這樣來的
WHERE / HAVING 子句	<code>where _carrier.carrname = 'Lufthansa'</code>	可以用關聯表的欄位當篩選條件，這時候一定會觸發 Join（因為要篩選就一定要讀那張表）
Filter Condition（[ ] 語法）	<code>association ... to b as _b on ... { } 搭配 _b[status = 'A'].field</code>	縮小關聯範圍，只考慮符合特定條件的關聯資料列（進階用法，這一課不要求會寫，知道有這個機制即可）

為什麼要這樣設計（用途 / 好處）：

1. 延遲決定，不強迫每個消費者都付 Join 的成本——具體展開講：

假設一個 Interface View 一次宣告了三個 Association（示意，不是真的要你寫）：`abap define view ZI_RAP02_TASK_EXT as select from zrap02_task association [0..1] to i_user as _createdByUser on zrap02_task.created_by = _createdByUser.username association [0..1] to`

zrap02\_dept as \_department on zrap02\_task.dept\_id = \_department.dept\_id association [0..\*] to zrap02\_hist as \_history on zrap02\_task.task\_id = \_history.task\_id { key task\_id, description, \_createdByUser, \_department, \_history } `` 這個 View 的欄位清單雖然把三個 Association 都「曝光」出來了，但實際會不會下推成 SQL 的 Join，要看每一次呼叫端的查詢怎麼寫：

- 呼叫端 A：SELECT task\_id, description FROM ZI\_RAP02\_TASK\_EXT INTO TABLE @DATA(lt\_a).——完全沒提到任何 \_xxx，最終送到資料庫的 SQL 等同於 SELECT task\_id, description FROM zrap02\_task，三個 **Association** 一個都沒被 Join，就好像它們根本不存在。
- 呼叫端 B：SELECT task\_id, description, \_department.dept\_name FROM ZI\_RAP02\_TASK\_EXT INTO TABLE @DATA(lt\_b).——只用到 \_department，最終 SQL 只會 Join zrap02\_dept 這一張表，\_createdByUser / \_history 兩個完全沒被碰到。
- 呼叫端 C：SELECT task\_id, \_createdByUser.fullname, \_history FROM ZI\_RAP02\_TASK\_EXT INTO TABLE @DATA(lt\_c).——這次換成用到 \_createdByUser 跟 \_history，SQL 就會 Join 這兩張表，\_department 反而完全沒被碰到。

重點是：這不是資料庫的 **Optimizer** 事後判斷「要不要用」，而是 **ABAP SQL 編譯器**在編譯這段 **SELECT** 語句當下，就已經根據你「有沒有寫 \_xxx」直接決定要不要把對應的 Join 子句生出來——沒被引用的 Association，連 Join 子句本身都不會出現現在送給資料庫的 SQL 裡，不是「Join 了但資料庫聰明地跳過」。這跟寫死的 inner join / left outer join 完全不同：寫死的 Join 一定會出現在 SQL 裡，就算你後面完全沒選任何被 Join 進來的表的欄位，那個 Join 子句還是在，資料庫還是要花力氣執行它（能不能被資料庫的「Join 消除」機制事後優化掉，要看資料庫實作、而且對 Outer Join / 一對多這種情境常常無法安全消除）——Association 則是從語言層面直接保證「沒用到就不會生成」，不用賭資料庫夠不夠聰明。

#### 1. 同一個宣告可以被重複利用——具體展開講：

延續上面的 ZI\_RAP02\_TASK\_EXT（裡面已經宣告好 \_department 這個 Association，on zrap02\_task.dept\_id = \_department.dept\_id 這段關聯邏輯只寫在這一個地方）。現在假設有兩個完全不同用途、由不同人建立的下流 CDS View，都疊在這個 Interface View 之上：

```
``abap " 消費端 1：給行動裝置用的精簡清單，完全不需要部門資訊 define view ZC_RAP02_TASK_LIST as select from ZI_RAP02_TASK_EXT { key task_id, description, status } ``abap " 消費端 2：給管理者看的報表，需要顯示部門名稱 define view ZC_RAP02_TASK_MGMT as select from ZI_RAP02_TASK_EXT { key task_id, description, status, _department.dept_name as department_name } ``
```

注意消費端 2 完全不需要知道、也不需要重寫 zrap02\_task.dept\_id = \_department.dept\_id 這段關聯條件——它只要寫 \_department.dept\_name，就直接繼承了 ZI\_RAP02\_TASK\_EXT 裡已經定義好的正確關聯邏輯。這才是「重複利用」真正的意思：關聯邏輯（要 Join 哪張表、用什麼欄位對應）只在 Interface View 這一層寫一次，之後不管有多少個下流 View、多少個 ABAP 程式要用到部門資訊，都直接引用 \_department，不用每個地方各自重寫一次 Join 條件。

這帶來兩個實際好處：

- 維護成本集中在一個地方：如果哪天部門資料的關聯邏輯要改（例如 zrap02\_dept 換了主鍵結構、或要多加一個「只看啟用中部門」的過濾條件），只要改 ZI\_RAP02\_TASK\_EXT 這一個地方的 Association 宣告，ZC\_RAP02\_TASK\_LIST、ZC\_RAP02\_TASK\_MGMT，以及未來任何新建的下流 View，全部自動套用新邏輯，不用一個一個去改。

- 每個消費端各自獨立決定要不要用：ZC\_RAP02\_TASK\_LIST 選擇完全不碰 `_department` ( 呼應第 1 點，它的 SQL 不會有任何部門相關的 Join )，ZC\_RAP02\_TASK\_MGMT 選擇要用——兩者共用同一個 Association 宣告，但各自的查詢效能完全不互相影響。

對比一下如果沒有 Association、只能用寫死的 Join 會怎樣：ZC\_RAP02\_TASK\_MGMT 就得自己重新寫一次 `inner join zrap02_dept on zrap02_task.dept_id = zrap02_dept.dept_id`——如果系統裡有 20 個地方都需要「任務 + 部門名稱」這種組合，就要在 20 個地方各自寫一次一模一樣的 Join 條件，之後要改關聯邏輯就要找過這 20 個地方逐一修改，很容易漏改或改到不一致。

1. 是 **OData \$expand / RAP Composition** 的技術基礎——Fiori Elements 畫面上常見的「展開明細列」「Header 帶出 Item 清單」，底層對應的就是把 Association 整個曝光在欄位清單裡（不指定欄位），讓 OData 消費端可以用 `$expand` 語法一次拿到巢狀資料；rap08 會用到的 `composition [0..*] of` ( Header-Item 結構 ) 也是同一套機制的特殊化版本。

這一課的練習 2 只要求你做到第一種用法（欄位清單裡用 `_carrier.carrname` 曝光單一欄位），後面幾種是進階用法，先知道有這回事就好，不要求現在就會寫。

## Built-in Functions ( 內建函數 )

CDS 提供跟 Open SQL 類似的內建函數，可以直接寫在欄位清單裡算出衍生欄位，常用的幾類：

分類	常用函數	範例
字串	<code>concat / substring / length / upper / lower / trim / replace</code>	<code>concat( task_id, description )</code>
數值	<code>abs / round / division / ceil / floor</code>	<code>round( amount, 2 )</code>
日期 時間	<code>date_add_days / date_days_between / date_is_valid</code>	<code>date_days_between( due_date, \$session.system_date ) as days_until_due</code>
條件	<code>case ... when ... then ... else ... end</code>	見下方範例

日期函數的例子剛好可以跟下一節的 Session Variable 串起來——算出「距離到期日還剩幾天」：

```
date_days_between($session.system_date, due_date) as days_until_due,

case status
 when 'O' then 'Open'
 when 'D' then 'Done'
 else 'Unknown'
end as status_text
```

## Parameters ( 參數 ) 與 Session Variables ( 系統變數 )

**CDS Parameters**：讓 View 在被查詢時可以「傳參數進去」，語法：



```

define view ZI_XXX
 with parameters
 p_status : zrap02_status
 as select from zrap02_task
{
 key task_id,
 description
}
where
 status = $parameters.p_status

```

參數在這個系統（沒有 **entity** 關鍵字的舊式語法）裡存取時：**p\_status** 或 **\$parameters.p\_status** 兩種寫法都可以（新式 **define view entity** 只能用 **\$parameters**。這種寫法，這是查證官方文件確認的差異）。呼叫端（ABAP SQL）要傳參數要用 **SELECT ... FROM ZI\_XXX( p\_status = '0' ) ...** 這種語法。

⚠ **p\_status : zrap02\_status** 這一行，冒號前後是兩個完全不同的東西，容易搞混：

```

with parameters
 p_distance : s_distance
" ↑ 這是「參數名稱」 ↑ 這是「參數的型別」

```

- 冒號前面（**p\_distance**）：參數名稱，是你自己取的，只要遵守識別字命名規則，取什麼都可以（不用是任何已存在的東西），呼叫端用 **SELECT ... FROM ZI\_XXX( p\_distance = 500 )** 或 View 內部用 **\$parameters.p\_distance / :p\_distance** 引用時，用的就是這個你自己取的名字。
- 冒號後面（**s\_distance**）：參數的型別，一定要是一個真實存在的 **DDIC 型別 / Data Element**（或 **abap.xxx** 這種內建型別），系統會拿這個型別去驗證你呼叫時傳進來的值合不合法（長度、資料型態）。⚠ 這裡最容易犯的錯，是把「底層表的欄位名稱」誤填成型別——例如 **SPFLI** 有一個欄位叫 **distance**，但這個欄位真正的型別是 **s\_distance**（Data Element），如果你把冒號後面寫成 **distance**（照抄欄位名稱），系統會去找一個叫「**DISTANCE**」的型別，但那個東西根本不存在（**DISTANCE** 只是欄位名稱，不是型別名稱），啟用會報 **Type DISTANCE is unknown** 之類的錯誤。要填的是這個欄位背後的 **Data Element** 名稱，不是欄位本身的名字——查一個欄位的 Data Element，可以直接讀那張表的 DDL 原始碼，冒號後面那個就是（例如 **SPFLI** 表裡 **distance : s\_distance not null;** 這一行，冒號後面的 **s\_distance** 才是要拿來當參數型別的值）。

⚠ ⚠ 一個很重要的觀念澄清：**Open SQL** 對 **CDS View** 下 **WHERE** 條件，不需要 View 有 **Parameter** 才能做——這件事 **Parameter** 跟 **WHERE** 是兩回事：

```

" 完全不需要 ZI_RAP02_TASK 有任何 Parameter，這樣寫一樣合法：
SELECT * FROM zi_rap02_task WHERE status = '0' INTO TABLE @DATA(lt_tasks).

```

呼叫端本來就可以對任何 CDS View 的輸出欄位（也就是欄位清單裡列出來的那些欄位）自由下 **WHERE** 條件，這是 Open SQL 的基本能力，跟這個 View 有沒有宣告 **with parameters** 完全無關。

那 **Parameter** 真正在解決什麼問題？——上面這個範例其實選得不夠好：它讓 **p\_status** 篩選的 **status**，如果你把 **status** 也放進欄位清單，呼叫端直接下 **WHERE status = '0'** 效果一模一樣，根本不需要 **Parameter**。**Parameter** 真正必要、**WHERE** 做不到的情境，是「這個值要影響 View 自己內部的計算邏輯或關聯

條件，而不只是篩選最終結果」——因為呼叫端的 **WHERE** 只能在 View 的邏輯全部跑完、產出結果列之後才介入篩選，沒辦法伸進 View 內部去影響「該怎麼算」或「該關聯到哪一筆」。舉個示意例子（不是真的要你寫，只是說明情境）：

```
" 假設有一張「依生效日期變動的價格表」zzprice，一個航班在不同時間點可能對應不同價格
define view ZI_XXX
with parameters
 p_reference_date : dats
as select from spfli
association [0..1] to zzprice as _price
 on _price.carrid = spfli.carrid
 and _price.valid_from <= $parameters.p_reference_date
 and _price.valid_to >= $parameters.p_reference_date
{
 key spfli.carrid,
 key spfli.connid,
 _price.amount as price_on_date
}
```

這裡 **p\_reference\_date** 決定的是「Join 的時候該接到 **zzprice** 的哪一筆」——這個決定必須在 Join 當下就做出來，呼叫端事後才下 **WHERE** 根本來不及（Join 已經接到某一筆或接不到了，結果已經定型），這種情境才是 Parameter 真正不可取代的用途。這一課練習 2 的 **p\_max\_distance** 其實用一般 **WHERE** 也做得到（因為 **distance** 本來就有列在輸出欄位裡），這裡用 Parameter 純粹是讓你練習語法，不是因為技術上非用不可——先誠實講清楚這一點，避免你誤以為「View 有 Parameter 才能篩選」。

⚠️⚠️ 再進一步澄清一個更根本的誤解：**Open SQL** 的 **WHERE** 不管有沒有搭配 **Parameter**，一律都會下推到資料庫執行，不會拉回 **Application Server** 才過濾——「Code to Data」這件事跟 **CDS View** 有沒有宣告 **Parameter** 完全無關：

- **WHERE**（呼叫端下的）：**Open SQL** 編譯這段 **SELECT ... WHERE ...** 陳述式時，會把 **WHERE** 條件直接組進送給資料庫的 SQL 語句裡——這是 **Open SQL** 最基本的行為，不是 **CDS** 特有的優化，從 **Classic ABAP**（甚至沒有 **CDS** 的年代，直接對 **Table** 下 **SELECT ... WHERE ...**）就是這樣運作，資料庫收到的就是一句完整帶 **WHERE** 的 SQL，篩選在資料庫端就做完了，不會把整張表的資料先撈回 **Application Server** 才在 **ABAP** 裡逐筆比對（那是完全不同的寫法，例如先 **SELECT \*** 全撈，再用 **LOOP AT ... WHERE ...** 或 **DELETE ... WHERE ...** 在 **ABAP** 記憶體裡處理，才是真正發生在 **Application Server** 的過濾，而且效能通常很差，這不是 **Open SQL WHERE** 的行為）。
- **Parameter**（傳進 View 的）：一樣是在組出送給資料庫的 SQL 語句的當下，就把參數值代入 View 內部的邏輯（**WHERE**、**ON** 條件.....），最終送到資料庫執行的還是同一句、已經帶入參數值的完整 SQL——一樣是 **Code to Data**，一樣在資料庫端執行。

所以兩者的差異不是「在哪裡執行」（兩者都在資料庫端執行），而是「能碰到什麼」：呼叫端的 **WHERE** 只能碰到 View 已經曝光出來的輸出欄位；Parameter 可以伸進 View 內部、影響輸出欄位「產生之前」的邏輯（哪個 Join 條件、哪個計算公式）。這才是這兩個機制真正的分工，不是「有沒有下推到資料庫」的差別。

在 **Eclipse ADT** 裡測試「帶 **Parameter** 的 **CDS View**」：前面（第 23 / 213 行）提過的 **Data Preview**，對這種有宣告 **with parameters** 的 View 一樣可以用，只是多一個步驟——因為這個 View 需要參數才能查詢，

Data Preview 會自動先跳出一個輸入參數值的畫面，輸入測試值後才會真的送出查詢。以練習 2 的 `ZI_RAP02_FLIGHT_PRACTICE` ( with parameters `p_distance : s_distance` ) 為例，操作順序：

- 1. 確認 View 已經啟用成功 ( `sap_inactive_objects` 或 Eclipse 裡沒有紅色錯誤標記 ) 。
- 2. 對著 `ZI_RAP02_FLIGHT_PRACTICE` 按右鍵 → **Open With** → **Data Preview** 。
- 3. 因為有宣告 Parameter，畫面不會直接顯示查詢結果，而是先跳出一個小視窗 / 欄位，列出這個 View 需要的參數名稱——這裡應該會看到 `P_DISTANCE` 這個輸入欄位 ( 是空的，等你填值 ) 。
- 4. 在 `P_DISTANCE` 欄位輸入一個測試值，例如 `500` ( 因為 `WHERE distance >= $parameters.p_distance` 這樣寫，`500` 代表「只看距離至少 500 的航班」)，按確認 / Enter 。
- 5. Data Preview 才會真正送出查詢，畫面下方會出現結果列表，欄位對應到 View 的 `SELECT` 清單：

| CARRID | CONNID | DISTANCE | ROUTE | CARRNAME | |---|---|---|---|---| | AA | 0017 | 2434 | JFKLAX |  
American Airlines | | LH | 0400 | 3968 | FRAJFK | Lufthansa | | ... | ... | ... | ... | ... |

( 以上是示意用的假資料，實際欄位名稱要看你 `as` 取的別名——這一課範例用 `Router` / `carrname` 這種別名，欄位標題會照你取的名字顯示；實際結果筆數跟內容要看系統裡 `SPFLI` 真實資料而定，如果輸入的門檻太高，結果可能是空的，屬於正常情況，不是錯誤。 )

- 1. 想換一個參數值重測，通常可以直接在 Data Preview 工具列找到「重新輸入參數」的按鈕 / 圖示，不用整個重開。

⚠ 這個流程目前是依 **Data Preview** 一般行為推測寫的，還沒有針對這一課的 `ZI_RAP02_FLIGHT_PRACTICE` 實際操作驗證過——如果你操作時畫面跟這裡描述的不一樣 ( 例如參數輸入畫面的位置、按鈕名稱 )，回報給我，我會照你實際看到的畫面修正這段說明。

**Session Variables** ( 系統內建，不用自己宣告，`$session.` 前綴 )：

變數	內容	對應 ABAP 系統欄位
<code>\$session.client</code>	目前 Client	<code>sy-mandt</code>
<code>\$session.user</code>	目前使用者	<code>sy-uname</code>
<code>\$session.system_language</code>	目前登入語言	<code>sy-langu</code>
<code>\$session.system_date</code>	系統日期	<code>sy-datum</code>
<code>\$session.user_date</code>	使用者所在時區的日期	<code>sy-datlo</code>
<code>\$session.user_timezone</code>	使用者時區	<code>sy-zonlo</code>

範例：拿這一課的 `ZI_RAP02_TASK` 示範兩種常見用法——一個是拿系統日期算衍生欄位 ( 呼應前面 Built-in Functions 教過的 `date_add` )，一個是拿目前使用者當篩選條件：

```
define view ZI_RAP02_TASK_WITH_SESSION
as select from zrap02_task
{
 key task_id,
 description,
 status,
 due_date,
```

```

created_by,

" 用法 1：拿系統日期算「距離到期日還剩幾天」，不用呼叫端額外傳日期進來
 dats_days_between($session.system_date, due_date) as days_until_due
}
where
" 用法 2：只看「我自己建立的」任務，不用呼叫端額外傳使用者名稱進來
 created_by = $session.user

```

兩個用法的共同重點：**`**$session.*`** 是系統自動提供的值，不需要（也不能）像 **Parameter** 那樣由呼叫端傳入——你不用寫 **`SELECT ... FROM ZI_RAP02_TASK_WITH_SESSION( ... )`** 額外帶參數，**View** 啟用後，**`$session.system_date / $session.user`** 這些值在每一次查詢當下\*\*都會自動填入「執行這個查詢當下」的系統日期跟目前登入者。也因為這樣，同一個 **View**、同一段程式碼，不同人在不同天執行，查到的結果會不一樣（**`days_until_due`** 會隨著今天的日期變動、**`created_by = $session.user`** 會隨著誰在執行而篩出不同的任務）——這是 **`$session`** 變數天生會有的「隨執行當下環境變動」特性，設計 **View** 時要意識到這一點，不要誤以為 **View** 的查詢結果永遠固定不變。

前面已經講過的 **CDS View Client Handling** 自動機制，背後用的就是 **`$session.client`**（**HANA** 那層對應的 **Session Variable** 叫 **`CDS_CLIENT`**）——這裡把伏筆補上，你現在知道那個「自動過濾」背後具體是靠什麼機制做到的了。

## 練習 2：自己建一個帶 **Association** 的 **CDS View**

**輪到你了**：用標準示範資料表 **`SPFLI`**（航班基本資料）跟 **`SCARR`**（航空公司）練習建一個全新的 **CDS View**（物件名稱、套件 **`$TMP`**，命名自訂，例如 **`ZI_RAP02_FLIGHT_PRACTICE`**），要求至少包含：

1. 一個 **Association**（不是 **Join**）：從 **`SPFLI`** 關聯到 **`SCARR`**（依 **`carrid`** 欄位），並透過 **path expression** 曝光 **`SCARR`** 的航空公司名稱欄位
2. 至少一個 **Built-in Function** 算出來的計算欄位（例如用 **`concat`** 把出發城市跟目的城市接成一個字串）
3. 至少一個 **Parameter**（例如篩選飛行距離門檻，用 **`SPFLI`** 的 **`distance`** 欄位）

建好、啟用成功後跟我說一聲（貼程式碼或截圖都可以），我會幫你核對語法有沒有踩到這個系統的已知坑（例如 **`entity`** 關鍵字、**`$session / $parameters`** 混用之類）。

這個練習的 **Association** 該用哪個 **Cardinality**？答案是 **`[0..1]`**——判斷過程：

- 先看方向：**`SPFLI`**（一筆代表一個航班班次）→ **`SCARR`**（一筆代表一家航空公司），這是「多筆 **`SPFLI`** 對一筆 **`SCARR`**」的方向（因為很多班次可能屬於同一家航空公司），所以是 **to-one**（**`[0..1]`** 或 **`[1..1]`**），不是 **`[0..*]`** / **`[1..*]`**——如果搞反成 **`SCARR`** → **`SPFLI`**，就要換成 **to-many**。
- 再看 **`[0..1]`** 還是 **`[1..1]`**：查過 **`SPFLI`** 的表定義，**`carrid`** 欄位確實有 **with foreign key ... scarr**，理論上「這個航班一定有對應的航空公司」，感覺應該用保證存在的 **`[1..1]`**。但這裡刻意選用比較保守的 **`[0..1]`**，原因呼應前面教過的重點：**DDIC** 外鍵只在畫面輸入層級生效，不是資料庫層的強制約束——**Open SQL** 的 **`INSERT`** 完全可以塞一筆 **`carrid`** 是 **`SCARR`** 裡沒有的航班進去，繞過這個檢查。既然「一定存在」這件事在資料庫執行期沒有真正的保證，用比較保守、不假設一定有資料的 **`[0..1]`** 會更安全；真的宣告成 **`[1..1]`** 卻遇到例外資料（航班存在、航空公司卻被刪了）時，行為反而可能不可預期。

## Part B：UI Annotation 語法入門——兩種寫法

.claude/rules/sap-adt-mcp.md 第 40.10 節已經查證過：`@UI.*` Annotation 跟「Classic RAP / ABAP Cloud RAP」這條語法版本軸線無關，這個系統完全支援，只是外層包裹的語句關鍵字（跟 CDS View 的 `define root view` vs `define root view entity` 一樣，如果是用寫法②）用的是舊式的 `annotate view`，不是新式的 `annotate entity`。

`@UI.*` Annotation 有兩種寫法，效果完全一樣，差別只在「寫在哪個檔案裡」：

#### 寫法①：直接寫在 CDS View 裡 ( Inline Annotation )

CDS 的 Annotation 語法本來就可以直接掛在 `define view` 陳述式上——Entity 層級的標記寫在 `define root view` 之前，欄位層級的標記寫在每個欄位前面，這個位置規則跟你在 Part A 已經寫過的 `@AbapCatalog.*` / `@AccessControl.*` 這些 annotation 完全一樣，`@UI.*` 沒有任何特殊之處，只是眾多 annotation 家族裡的一種，一樣可以直接混在同一份 CDS View 原始碼裡，不一定要另外開檔案。例如把 rap02 的 `ZI_RAP02_TASK` 直接加上 UI 標記，會長這樣：

```
@AbapCatalog.sqlViewName: 'ZIRAP02TASK'
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'RAP02 Task Interface View'
@ObjectModel.compositionRoot: true
@UI: {
 headerInfo: {
 typeName: 'Task',
 typeNamePlural: 'Tasks',
 title: { type: #STANDARD, value: 'description' }
 }
}
define root view ZI_RAP02_TASK
 as select from zrap02_task
{
 @UI.lineItem: [{ position: 10 }]
 @UI.selectionField: [{ position: 10 }]
 key task_id,

 @UI.lineItem: [{ position: 20 }]
 description,

 @UI.lineItem: [{ position: 30 }]
 @UI.selectionField: [{ position: 20 }]
 status,

 @UI.lineItem: [{ position: 40 }]
 priority,

 @UI.lineItem: [{ position: 50 }]
 due_date,
```



```
created_at,
created_by
}
```

跟下面寫法②的完整範例對照一下就會發現：`@UI.headerInfo` / `@UI.lineItem` / `@UI.selectionField` 這幾個標記的寫法、屬性、值完全一模一樣，唯一的差別只是「這次直接混在 `ZI_RAP02_TASK` 這份檔案裡」，不需要另外建一個物件。這個範例故意不寫 `@Metadata.allowExtensions: true`——這個 annotation 的作用是「允許之後另外有一份獨立的 Metadata Extension 疊加上來」，只有你打算走寫法②（或想讓別人事後用寫法②擴充）才需要它，純粹寫法①、不打算讓任何人事後疊加 Metadata Extension 的話，這一行可以省略。

**@UI.headerInfo** 這一段的用途：這是 Entity 層級的標記（管的是整個實體，不是某個特定欄位），控制 Fiori Elements 在幾個「代表這個實體本身」的地方要顯示什麼文字——沒有這段標記，畫面會退回顯示 CDS View 的技術名稱這種不友善的預設值：

- **typeName: 'Task'**：這個實體的單數顯示名稱。用在 Object Page（單筆詳細畫面）標題區域「這是一個 Task」這種語意的地方，例如頁面上方會出現 Task 這個字樣標示「你正在看的是一筆 Task 類型的資料」。
- **typeNamePlural: 'Tasks'**：這個實體的複數顯示名稱。用在 List Report（清單畫面）的標題，例如清單上方會顯示 Tasks (3) 這種「類型名稱 + 筆數」的組合，3 是目前查到的筆數，Tasks 就是這裡設定的值。
- **title: { type: #STANDARD, value: 'description' }**：告訴 Fiori Elements「這筆資料的『標題』要用哪個欄位的值」——因為技術主鍵（`task_id`，可能是一串沒有意義的編號）通常不適合直接當標題給使用者看，這裡設定成用 `description` 欄位的值當標題，Object Page 打開某一筆資料時，頁面最頂端的大標題文字就會直接顯示這筆資料的 `description` 內容（例如「準備教材」），而不是顯示 `task_id` 的編號。**type: #STANDARD** 是最常見的寫法，代表「直接取某個欄位的值當標題」；還有其他進階的 **type**（例如把多個欄位組合成一段文字）但這一課用不到。

簡單說：拿掉這一整段 `@UI: { headerInfo: {...} }`，程式一樣能啟用、資料一樣能顯示，只是畫面上「這是什麼類型的資料」跟「這一筆資料該怎麼稱呼」會變得很技術性、不好閱讀——這正是 UI Annotation 的本質：不影響資料對不對，只影響「人看畫面時好不好懂」。

這個寫法的優點：不用多開一個物件、少一層要維護的檔案，資料模型跟 UI 顯示邏輯集中在同一個地方一目了然，小範圍練習或單人維護的專案很方便。

寫法②：獨立成 **Metadata Extension**（這一課實際採用的做法）

為什麼這一課選擇寫法②，不用上面更簡單的寫法①：`@UI.*` 這類 Annotation 純粹是「這個欄位在 Fiori Elements 畫面上怎麼顯示」，跟資料模型本身無關。拆到 Metadata Extension（DDLX，一個跟 **CDS View** 同名、但物件型別不同的獨立物件）的好處是：核心資料模型維持乾淨，UI 客製化可以獨立維護、甚至以後讓不同團隊/國家疊加各自的畫面客製化而不用改到共用的 View 本體——這在你自己就是 **View** 的作者時是「要不要多開一個檔案」的取捨；但如果這個 View 不是你寫的（例如標準 SAP CDS View，或別的團隊維護的 View，你沒有修改權限），寫法①根本行不通，**Metadata Extension** 是唯一能加 UI 標記的辦法，這也是為什麼實務上（尤其 SAP 標準內容）幾乎都用寫法②，很少看到寫法①。

⚠️「跟 **CDS View** 同名」是慣例，不是強制規定——查證官方文件（Creating CDS Metadata Extensions）確認：Eclipse 建立 Metadata Extension 的精靈裡，**Name**（這個 **DDLX** 物件自己的名稱）跟 **Extended Entity**（它要擴充哪一個 **CDS View**）是兩個完全獨立的欄位，理論上你可以把 DDLX 取名叫 `ZFOO_BAR`，內容卻是

`annotate view ZI_RAP02_TASK with { ... }`——真正決定「這份 Metadata Extension 是在擴充哪個 View」的，是 `annotate view <名稱> with` 這一行明確寫出來的 **View 名稱**，不是 DDLX 物件自己叫什麼。

那為什麼幾乎所有教材 / 範例都用同名：

1. 一眼就看得出對應關係——物件清單裡看到 `ZI_RAP02_TASK` (DDLX/EX)，不用打開來看內容就知道它是在擴充哪個 View；取不同名字的話，要打開檔案讀 `annotate view` 那一行才知道。
2. 實務上通常就是 1 對 1 關係——同一個 CDS View，同一個「客製化分層」( `@Metadata.layer`，例如都是 `#CUSTOMER`) 不能有兩份 **Metadata Extension** 同時擴充它 (會衝突)，所以「一個 View 對一個 DDLX」在實務上本來就是常態，用同名把這個對應關係直接體現在物件名稱上，是最直覺的做法。
3. Eclipse 精靈如果你是從 **CDS View** 物件右鍵新增 Metadata Extension (而不是從空白套件新增)，通常會自動帶出跟 View 相同的名稱當預設值——這也是為什麼大部分人 (包含這門課) 幾乎不會特意取不同的名字，順著精靈預設值走就好。

**結論：**技術上不強制同名，但建議你這一課、以及之後的練習，都照著同名慣例走——沒有不這樣做的好理由，而且能避免自己之後回頭看物件清單時搞不清楚對應關係。

## Eclipse ADT 建立 Metadata Extension : Step by Step

跟建 Table / CDS View 一樣，先講怎麼從零建出一個 Metadata Extension 空殼 ( ☒ 已依使用者實測畫面校正 )：

1. 對著你要擴充的 CDS View (例如 `ZI_RAP02_TASK`) 按右鍵，選單裡直接有 **New Metadata Extension** 這個項目 (跟這門課一路遇到的其他物件一樣，不需要繞經 **Other ABAP Repository Object**)。
2. 填寫：**Package** ( `$TMP` )、**Name**、**Description**、**Extended Entity**——從 CDS View 右鍵新增時，**Name** 跟 **Extended Entity** 通常會自動帶出跟這個 View 相同的名稱 (呼應前面「同名慣例」那一段)，確認無誤後按 **Next**。
3.   接著會跳出「**Templates**」畫面，這一步選錯會直接踩到這系統的已知語法限制——畫面預設可能停在 **Annotate Entity (creation)** 這個分類 (底下有 `annotateEntity` / `annotateEntityWithParameters` 兩個模板)，這個分類產生的骨架是新式語法：

```
``abap @Metadata.layer: ${layer} annotate entity ${entity_name} with { ${element_name};
${cursor} }`
```

注意是 `annotate entity` (帶 `entity` 關鍵字)——這系統不支援，跟 **CDS View** 精靈裡「**Define View Entity**」模板不能用是同一個限制。要展開下面收合的 **Annotate View (creation)** 分類，改選那裡面的模板 (產生的骨架應該是不帶 `entity` 的 `annotate view`)，跟這一課教的語法一致才選對。選好後 **Next** → `$TMP` 的 Transport 畫面直接 **Finish**。

1. 編輯器開啟後，會看到類似這樣的骨架 (兩個地方預設值都不是可以直接用的最終內容，需要你自己填)：

```
``abap @Metadata.layer: layer annotate view ZI_RAP02_TASK with { element_name; } ``
```

- **@Metadata.layer: layer**——**layer** 是一個佔位符文字，不是合法的列舉值，一定要改成下面列出的其中一個真正的列舉值 (這一課固定用 `#CUSTOMER`)。
-  骨架預設沒有 **@UI: { }** 這個 **Entity 層級 Annotation 區塊**——如果你需要 `headerInfo` 這類 Entity 層級設定 (本課一定需要)，要自己在 `@Metadata.layer` 那一行後面手動加上整段 `@UI: { headerInfo: {...} }`，精靈不會幫你生出來。
- 把骨架改成本課要的完整內容 (見下方完整程式碼)，**Ctrl+S** 存檔 → **Activate**。

**@Metadata.layer** 的完整可選列舉值（查證官方語法文件確認）：

列舉值	意義
#CORE	SAP 原廠自己交付的標準內容用——客戶 / Partner 開發不會用到
#LOCALIZATION	SAP 針對特定國家 / 地區在地化需求交付的內容用
#INDUSTRY	SAP 針對特定產業解決方案交付的內容用
#PARTNER	SAP 合作夥伴（例如做加值方案的 ISV）交付的擴充用
#CUSTOMER	客戶自己客製化的內容——這門課、以及一般企業自己開發的 Z 物件一律用這個，其他四個都是 SAP 生態系裡不同角色專用，不會是我們會用到的值

這五個值的存在，呼應前面「為什麼要獨立建一個物件」提過的重點：同一個 **CDS View** 理論上可以同時被 **SAP 原廠**（**#CORE**）、**在地化套件**（**#LOCALIZATION**）、**產業方案**（**#INDUSTRY**）、**合作夥伴**（**#PARTNER**）、**客戶自己**（**#CUSTOMER**）各自疊加一份 **Metadata Extension**，系統靠這個標記知道每一份分別是誰寫的、合併顯示時該用什麼優先順序疊加——但同一層（例如兩份都宣告 **#CUSTOMER**）疊加到同一個 View 會衝突，這也是為什麼前面說「實務上通常是 1 對 1 關係」。

整體語法結構總覽（先看這個，再看下面的逐行細節）

一份 DDLX 檔案固定分成三個部分，由上到下依序排列：

```
@Metadata.layer: #CUSTOMER " ① 檔案層級 Annotation，只出現一次，寫在最上面

@UI: { " ② Entity 層級 Annotation（選擇性，看你需不需要）
 headerInfo: { ... } " 寫在 annotate view 之前，因為它不屬於任何單一欄
 位
}

annotate view ZI_RAP02_TASK with " ③ 真正的「附加」語句，一個 DDLX 檔案只會有一句
{
 @UI.lineItem: [{ position: 10 }] " 每個欄位前面加 Annotation
 task_id; " 欄位名稱 + 分號結尾
 ...
}
```

這個結構是固定的，不能隨便調換順序——**@Metadata.layer** 一定在最上面；任何 Entity 層級（不屬於特定欄位）的 Annotation（**@UI.headerInfo** / **@UI.facet** 這類）一定要寫在 **annotate view** 之前；**annotate view <CDS View 名稱> with { }** 這句本身整份檔案只會出現一次，因為一個 DDLX 只對應一個 CDS View。

這個大括號 **{ }** 的格式是 **JSON** 嗎？——長得像，但不是真的 **JSON**，是 ABAP CDS 自己定義的「Annotation Value」語法，借用了 JSON 的巢狀 **{ }** / **[ ]** 概念，但幾個地方跟標準 JSON 不一樣，不能直接拿 JSON 語法規則硬套：

- 字串用單引號 **'...'**（ABAP 慣例），標準 JSON 一定要雙引號 **"..."**

- 列舉值用 `#STANDARD` 這種 `#` 開頭的寫法 ( 例如 `title: { type: #STANDARD, ... }` ) , 標準 JSON 沒有這種語法 , 只能寫成字串 `"STANDARD"`
- 物件裡的 Key ( 例如 `headerInfo: 、 typeName:`  ) 不用加引號 , 這點比較接近 JavaScript 物件字面量 ( Object Literal ) 而不是嚴格 JSON ( JSON 規範要求 Key 一定要用雙引號包起來 )

三個部分各自的目的 :

- ① `@Metadata.layer: #CUSTOMER` : 宣告這份 Metadata Extension 屬於哪一個「客製化分層」。SAP 的架構允許同一個 CDS View 疊加多份不同來源的 Metadata Extension ( SAP 原廠一份、Partner 外掛一份、你自己客製一份 ) , 這個標記讓系統知道「這份是誰寫的、疊加時該用什麼優先順序合併」—— `#CUSTOMER` 就是「這是客戶自己寫的客製化」, 我們課程一律用這個值就好, 不用理會其他選項 ( `#CORE` / `#LOCALIZATION` / `#PARTNER` 是 SAP 原廠或合作夥伴專用 ) 。
- ② `@UI: { headerInfo: { ... } }` : Entity 層級的 UI 設定——「Entity 層級」的意思是這個 Annotation 管的是整個 **View** ( 例如「這個實體顯示的標題」) , 不是某一個特定欄位, 所以放不進 `annotate view ... with { }` 那個逐欄位的區塊裡, 只能獨立寫在外面。目前這一課只用到 `headerInfo` , 之後如果要加其他 Entity 層級設定 ( 例如上一節提過的 `@UI.facet` ) , 也是寫在這個位置。
- ③ `annotate view ZI_RAP02_TASK with { ... }` : 真正做事的語句——把後面 `{ }` 裡列出的每個欄位, 各自掛上你寫的 `@UI.xxx` 標記。這個區塊只需要列出「你想加 UI 標記的欄位」, 不用把 CDS View 的每個欄位都列進來 ( 前面已經提過: 沒被列進來的欄位, 就是「沒有 UI 標記」的預設狀態, 不會出錯 ) 。

⚠ `annotate view <名稱> with` 這個 `<名稱>` 一定要填對, 而且是 **DDL View Name** , 不是 **SQL View Name** :

- 一定要正確對應到一個真實存在、已啟用的 **CDS View**——如果填的名稱拼錯、或指到一個根本不存在的 View , 啟用會直接失敗 ( 系統找不到要擴充的對象 ) ; 如果指到一個存在、但目前是 Inactive ( 未啟用 ) 的 View , 通常也會啟用失敗或至少產生警告, 因為 Metadata Extension 依附的對象本身還不穩定。
- 這裡填的是 **DDL View Name** , 不是 **SQL View Name**——呼應前面查證過的規則 ( Open SQL / Eclipse ADT / quickSearch 這類 CDS 之後才有的現代化管道一律用 DDL Name , 只有 SE11 View 畫面 / SQ01 / SQ02 這類比 CDS 更早存在的傳統工具才認 SQL View Name ) : `annotate view` 本身就是一句 **CDS DDL 語句** ( `ANNOTATE ENTITY cds_entity WITH ...` , 官方語法文件裡這個位置的參數就叫 `cds_entity` , 指的是 CDS 邏輯層的實體 ) , 所以理所當然只認 DDL Name。這一課的範例填的是 `ZI_RAP02_TASK` ( DDL Name ) , 如果誤填成 `ZIRAP02TASK` ( SQL View Name , `@AbapCatalog.sqlViewName` 那個值 ) 會直接找不到對應的 View、啟用失敗。

```
@Metadata.layer: #CUSTOMER
@UI: {
 headerInfo: {
 typeName: 'Task',
 typeNamePlural: 'Tasks',
 title: { type: #STANDARD, value: 'description' }
 }
}

annotate view ZI_RAP02_TASK with
{
```

```

@UI.lineItem: [{ position: 10 }]
@UI.selectionField: [{ position: 10 }]
task_id;

@UI.lineItem: [{ position: 20 }]
description;

@UI.lineItem: [{ position: 30 }]
@UI.selectionField: [{ position: 20 }]
status;

@UI.lineItem: [{ position: 40 }]
priority;

@UI.lineItem: [{ position: 50 }]
due_date;
}

```

三個基礎 **@UI.\*** 標記，這一課只需要記住它們的角色分工：

- **@UI.headerInfo**：Entity 層級（寫在 **annotate view ... with** 外面），控制 Object Page（單筆詳細畫面）的標題怎麼顯示——這裡設定「顯示名稱是 Task/Tasks」「標題文字取 **description** 欄位的值」
- **@UI.lineItem**：欄位層級，標記這個欄位要不要出現在 List Report（清單）的表格欄位裡，**position** 決定欄位出現的順序
- **@UI.selectionField**：欄位層級，標記這個欄位要不要出現在清單畫面的篩選/搜尋條件裡

這兩個標記的「值」為什麼是 **[{ ... }]**（中括號包大括號），裡面還能放什麼：

```

@UI.lineItem: [{ position: 10 }]
@UI.selectionField: [{ position: 10 }]
task_id;

```

- 外層的中括號 **[ ]** 代表「陣列」——同一個欄位理論上可以同時屬於多組不同的 List Report / Selection 設定（例如給不同畫面變體各自客製化順序），所以語法上一律是陣列，即使像這一課只放一組設定，還是要用 **[{ ... }]** 包起來，不能只寫 **{ ... }**（少了中括號會啟用失敗）。
- 陣列裡的每個 **{ }** 是一組設定，除了這一課用到的 **position**（數字，決定順序），還有幾個之後可能會遇到但這一課不要求你會寫的常用屬性（先知道有這回事就好，不用背）：

屬性	用在	作用
<b>position</b>	兩者都有	排序用的數字，越小排越前面——數值本身沒有特殊意義，純粹用來排序， <b>11 / 12 / 13</b> 這種連續數字一樣合法。範例都習慣用 <b>10 / 20 / 30</b> （間隔 10）是 ABAP 常見慣例，刻意留間隔方便以後在中間插入新欄位（例如之後想在 10 跟 20 中間加一個，直接給 <b>15</b> 就好，不用把後面的編號全部往後挪），不是規定，不留間隔也不會出錯



屬性	用在	作用
label	兩者都有	覆寫這個欄位在畫面上顯示的文字，不寫的話預設用 Data Element / @EndUserText.label 的說明文字
importance	@UI.lineItem	#HIGH / #MEDIUM / #LOW，畫面空間不夠（例如手機）時，系統會優先隱藏 importance 較低的欄位
type	@UI.lineItem	進階用法，例如把這個欄位顯示成按鈕（#FOR_ACTION）而不是純文字，這一課用不到

這一課只要求你會用 **position**，其他屬性是進階內容，之後規劃中的 Fiori Elements 課程才會深入教怎麼設計實際畫面版面。

⚠ 這一課實測踩到的語法規則：`annotate view ... with { }` 區塊裡列出的每個欄位，至少要有有一個 **Annotation**，不能空白列出——一開始把 `created_at` / `created_by` 也列進去但沒加任何標記，啟用時報 `Element 'created_at' must have at least one annotation`。修法很直接：這兩個欄位本來就不需要在畫面上曝光，直接不列進 `annotate view` 區塊即可（CDS View 裡沒被 Metadata Extension 提到的欄位，就是維持「沒有 UI 標記」的預設狀態，不會出錯）。

⚠ ⚠ 已依使用者實測更正：**Metadata Extension** 在 **SE11** 查不到，但 **SE80 (Object Navigator)** 可以，而且是完整顯示原始碼，不是只有基本資訊——原本猜測 SE80 大概只能查到「這個物件存在」這種基本資訊，被使用者實測推翻：

- SE80 的 Repository Browser，展開套件（\$TMP）之後，樹狀結構裡有一個專屬的 **Metadata Extensions** 節點，跟 **Behavior Definitions** / **Service Bindings** / **Service Definitions** 平行並列——代表 SE80 對 RAP 相關的現代物件型別（DDLX / BDEF / SRVD / SRVB）都有專屬分類，不是只認得 Table / Class 這些傳統物件。
- 點開 **ZI\_RAP02\_TASK**（Metadata Extension）會開啟「Display Metadata Extension」畫面，**Source Code** 頁籤直接完整顯示 **DDLX** 的原始碼（`@UI: { headerInfo: {...} }`、`annotate view ... with { ... }` 整段都看得到），是唯讀顯示（Display 模式），但內容完整不打折扣。
- 畫面上還有一個 **ADT Link** 欄位，顯示這個物件對應的 ADT URI（例如 `adt://S4H/sap/bc/adt/ddic/ddlx/sources/zi_rap02_task`）——SAP GUI 跟 Eclipse ADT 之間有做這種互相連結，方便從 SE80 一鍵跳轉到 Eclipse 開啟同一個物件編輯。
- 這個結論可以再擴大：既然 SE80 對 Metadata Extension 都能完整顯示，合理推測整個 **RAP 五層架構**（**Table** / **CDS View** / **BDEF** / **Service Definition** / **Service Binding**）在 **SE80** 應該都能用同樣方式唯讀瀏覽原始碼 / 設定——這一課實測畫面上剛好也同時看到 **Behavior Definitions**（**ZI\_RAP02\_TASK** / **ZI\_RAP03\_UMTEST** / **ZI\_RAPT01**）、**Service Bindings**（**ZRAP04\_SB** / **ZRAP01\_SB**.....）、**Service Definitions**（**ZRAP04\_SD** / **ZRAP01\_SD**）都列在同一棵樹裡，佐證了這個推測。結論：想快速「看一眼」某個 RAP 物件的內容，又不想開 **Eclipse**，**SE80** 是比 **SE11** 更好用的唯讀瀏覽工具（**SE11** 只認 DDIC / 資料庫層物件，**SE80** 對 Repository 物件的涵蓋範圍廣得多）；但真正要編輯內容，還是只能用 **Eclipse ADT**，SE80 這裡看到的都是唯讀顯示。

## 學習目標

- 能在 Eclipse ADT 完整走過一次「New Database Table」精靈，從空殼建出一張 Transparent Table
- 能設計一張適當 RAP 根實體的 DDIC Table（正確的 Key 結構、欄位型別選擇），分清楚 CDS 內建型別（`abap.*`）跟引用 Data Element（不加前綴）兩種寫法的差異

- 能寫出 `key / not null / Foreign Key` ( 含 cardinality ) 的正確語法，知道 DDIC 外鍵只在畫面輸入層級生效、不是資料庫強制約束
- 知道金額 / 數量欄位要用 `@Semantics.amount.currencyCode / @Semantics.quantity.unitOfMeasure` annotation 指定參考欄位，以及缺這個 annotation 在多個金額欄位共用貨幣欄位時會啟用失敗的原因
- 知道 Primary Index ( Key 自動產生 ) 跟 Secondary Index ( 額外加速結構，Optimizer 自行決定要不要用 ) 的差異；知道這系統的 Eclipse ADT 「New Table Index」精靈對長表名有已確認的 bug 走不通，要改用 SE11 建立；知道 Explain Plan 的概念 ( 就算索引建對了，Optimizer 也不一定會用 )，並知道這系統的 SQL Console 沒有這個功能
- 知道欄位型別一律要引用 Data Element 的硬性規則：語意對應標準表既有欄位就直接重用標準 DE ( 先用 quickSearch 查證，不要憑記憶猜 )，真的沒有合適標準 DE 才自己建 Domain + DE，不可以留著 `abap.char(...)` 這類內建型別偷懶
- 能寫出 Domain 固定值清單的建立語法，並知道固定值清單只在 UI 層生效 ( F4 下拉 )，不是資料庫層的強制約束，跟 Validation ( rap06 ) 的角色不同
- 能在 Eclipse ADT 完整走過一次「New Data Definition」精靈，從空殼建出一個 CDS View
- 能寫出這個系統適用的 CDS Interface View 語法 ( `define root view`，四個必要 annotation 各自的作用 )
- 知道 DDL View Name ( Eclipse / ABAP 程式碼裡用的邏輯名稱，上限 30 碼 ) 跟 `@AbapCatalog.sqlViewName` 指定的 SQL View Name ( 資料庫底層實體物件名稱，純技術用途，上限 16 碼 ) 是兩個不同名字、各自的用途
- 在 Eclipse ADT 的「New Data Definition」精靈裡，知道要選 **Define View (obsolete as of AS ABAP 7.57)** 這個模板，不要選帶 `View Entity` 字樣的模板 ( 這系統的 ABAP 編譯器不支援新式 `entity` 語法 )
- 知道為什麼 CDS View 不需要 ( 也不應該 ) 把 `client / mandt` 欄位列進欄位清單——CDS 的 Client Handling 機制會自動處理 ( 背後靠 `$session.client` )，跟 AMDP / SQLScript 「完全不自動處理 Client、要自己手動過濾」正好相反
- 理解欄位別名跟 `mapping for` 的取捨，知道什麼情況才需要用別名
- 知道 CDS View 的分類慣例 ( `I_ / C_或P_ / R_` )，能講出 RAP 的 Interface View 屬於哪一類、跟 Projection View 的分工
- 能講出 Association 跟 Join 的差異 ( Association 只有真的被 path expression 引用才會轉成 Join，沒用到不影響效能 )，知道這是 RAP Composition 語法的基礎
- 知道 CDS 內建函數的幾個常用分類 ( 字串 / 數值 / 日期時間 / 條件 )，能寫一個簡單的計算欄位
- 能寫出 CDS Parameters ( `with parameters` ) 語法，知道這系統支援 `:pname` 跟 `$parameters.pname` 兩種存取寫法；知道常用內建 Session Variable ( `$session.client / user / system_date` 等 )
- 能建立一個 Metadata Extension ( DDLX ) 物件，寫出這個系統適用的 `annotate view ... with { }` 語法
- 能寫出三個基礎 `@UI.*` 標記 ( `headerInfo / lineItem / selectionField` )，知道各自控制畫面的哪個部分
- 知道 Metadata Extension 裡列出的每個欄位都必須至少有一個 Annotation，沒有 UI 需求的欄位乾脆不要列進去

## 物件清單

物件	名稱	型別
DDIC Table	ZRAP02_TASK	TABL/DT

物件	名稱	型別
Domain	ZRAP02_TASKID / ZRAP02_STATUS / ZRAP02_PRIORITY / ZRAP02_DUEDATE	DOMA/DD
Data Element	ZRAP02_TASKID / ZRAP02_STATUS / ZRAP02_PRIORITY / ZRAP02_DUEDATE	DTL/DE
CDS Interface View	ZI_RAP02_TASK	DDL/DF
Metadata Extension	ZI_RAP02_TASK ( 同名，型別不同 )	DDLX/EX

全部物件都在 \$TMP 套件，已建立並啟用 ( `sap_inactive_objects` 確認 0 筆殘留 )。

延伸練習物件 ( 你自己動手建，名稱自訂，不算在上面正式的課程物件清單裡 )：

物件	建議名稱	型別	對應練習
Secondary Index	<表名>~001 ( SE11 建立，例如 ZRAP02_TASK1~001 )	INDX/DD	練習 1 ( 索引 + EXPLAIN PLAN，走 SE11 建立，不要用 Eclipse ADT 精靈——見上方確認過的 bug 記錄 )
CDS View ( 含 Association / Built-in Function / Parameter )	ZI_RAP02_FLIGHT_PRACTICE ( 自訂 )	DDL/DF	練習 2 ( Association vs Join )

## 驗證方式

Table / CDS View / Metadata Extension 都不是可執行的程式，沒有 `programrun` 這種無頭驗證手段，這一課的驗證重點是語法正確 + 成功啟用：

1. 用 `checkruns` API ( 見規則檔第 4 節 ) 或直接嘗試啟用，確認沒有語法錯誤
2. `sap_inactive_objects` 回傳空清單，代表沒有殘留的未啟用版本
3. 讀回 `version=active` 的原始碼，逐字比對跟預期一致

真正「畫面長什麼樣子」的驗證 ( Metadata Extension 的 `@UI.*` 標記實際效果 )，要等 rap04 建出 Service Binding、Publish 之後，用 Fiori Elements Preview 才看得到——這一課先確保語法正確、能啟用即可。

兩個延伸練習的驗證方式：

- 練習 1 ( **Secondary Index** )：SE11 建立成功、索引畫面顯示 **Status: Active / Saved** ( 這系統的 SQL Console 沒有 Explain Plan 功能，已確認，不列入驗收依據 )
- 練習 2 ( **CDS View** )：Eclipse 啟用成功 + 用 Data Preview 看查詢結果 ( 記得要傳入 Parameter 才查得到資料 )，貼程式碼給我核對語法

## 思考題

1. 如果之後想讓 `task_id` 由系統自動產生（不用使用者手動輸入），資料型別跟目前的 `abap.char(10)` 手動輸入模式相比，通常會怎麼設計？（提示：想想 RAP 常見的 GUID / Early Numbering 概念，這是 rap03 會碰到的話題）
2. `@UI.selectionField` 只加在 `task_id` 跟 `status` 兩個欄位，如果之後想讓使用者也能用 `priority` 篩選，要怎麼修改？
3. 如果同一個 CDS View 之後想給兩種不同的畫面用（例如一個給管理者看全部欄位、一個給一般使用者看精簡版），Metadata Extension「一個 View 只能對應一個」這件事，會不會是個限制？（提示：這就是為什麼真實專案常常會另外設計 Projection View，讓不同消費情境各自套用不同的 Metadata Extension——這個系統的單層模式簡化了這一步，多消費情境是進階主題）
- 4.（純觀念題，這系統驗證不了）如果幫 `ZRAP02_TASK` 建了一個 `status + due_date` 的 Secondary Index，在一個真的有 Explain Plan 功能的系統上分析，結果顯示 Optimizer 還是選擇全表掃描，這代表你的 Index 建錯了嗎？該怎麼進一步判斷？（提示：回顧「用 SQL Console 的 EXPLAIN PLAN 確認 Index 有沒有被用到」那一節列出的幾個常見原因）
  1. 練習 2 如果把 Association 換成 Join 硬接 `SPFLI` 跟 `SCARR`，即使你完全沒有在欄位清單引用 `SCARR` 的欄位，查詢效能會不會被影響？為什麼跟 Association 的行為不一樣？

## 答案

見 `zrap02_task.tabl.abap`、`zrap02_taskid.doma.xml / .dtel.xml`、`zrap02_status.doma.xml / .dtel.xml`、`zrap02_priority.doma.xml / .dtel.xml`、`zrap02_duedate.doma.xml / .dtel.xml`、`zi_rap02_task.ddls.abap`、`zi_rap02_task.ddlx.abap`。SAP 端物件：`ZRAP02_TASK`（Table）、四組同名 Domain / Data Element、`ZI_RAP02_TASK`（CDS View + Metadata Extension，同名不同型別）。兩個延伸練習（Secondary Index、帶 Association 的 CDS View）由你在 Eclipse 動手建立，沒有固定答案快照——建好後跟我核對即可。